Event Manager - Report

612 words

1. Introduction

The Event Manager is built on the supplied Express/EJS template and SQLite. In addition to the base requirement, the project introduces an extension that secures organiser pages with password authentication enforced by bcrypt hashing and express-session. The use of three tier architecture separates concerns clearly, allowing the browser to handle presentation, Express server handles logic, and SQLite persists data.

2. Architecture

Three tiers Architecture:

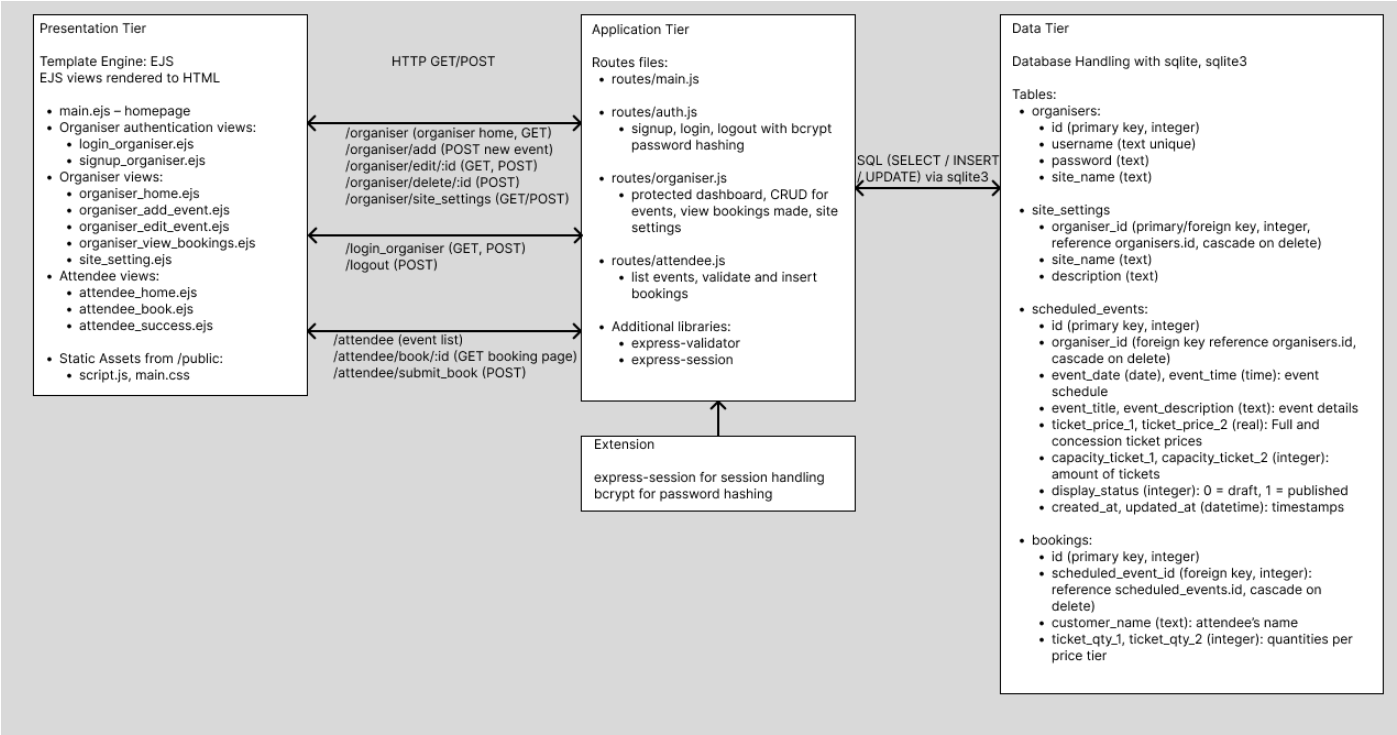

Figure 1 Three Tiers Architecture

Presentation Tier:

EJS templates are rendered to HTML and display dynamic data such as event titles and descriptions, with CSS and client-side JavaScript are served from the

/public folder allowing the browser to style and handle small interactions such as closing alerts without fetching data from the server.

Application Tier:

Node.js v22 and Express 4 power the route logic, with the libraries express-validator, express-session, and bcrypt. express-validator trims and validates form fields, express-session stores logged in organiser, and bcrypt hashes and verifies organiser passwords.

Files in the /routes folder control CRUD operations, authentication, capacity checks and session handling

Data Tier:

sqlite3 is used to interact with SQLite. There are four tables, organisers, site_settings, scheduled_events, bookings linked by cascading foreign keys.

The schematic diagram of the three tiers architecture shows:

- HTTP GET and POST requests from the browser and the corresponding responses from the server.
- SQL SELECT, INSERT, UPDATE flow between server and database.
- Authentication components (express-session, bcrypt) as the extension

End points that connect client to server

- Organiser:
    - GET /organiser, go to organiser home
    - POST /organiser/add, create a new draft event
    - GET/POST /organiser/edit/:id, edit and update event
    - GET/POST /organiser/delete/:id, delete event
    - GET/POST /organiser/site_settings, change site name and description.
- Attendee:
    - GET /attendee, go to attendee home
    - GET /attendee/book/:id, view booking form
    - POST /attendee/submit_book, submit booking
- Extension:
    - GET/POST /login_organiser, login as organiser
    - POST /logout, logout

3. **Data Model**

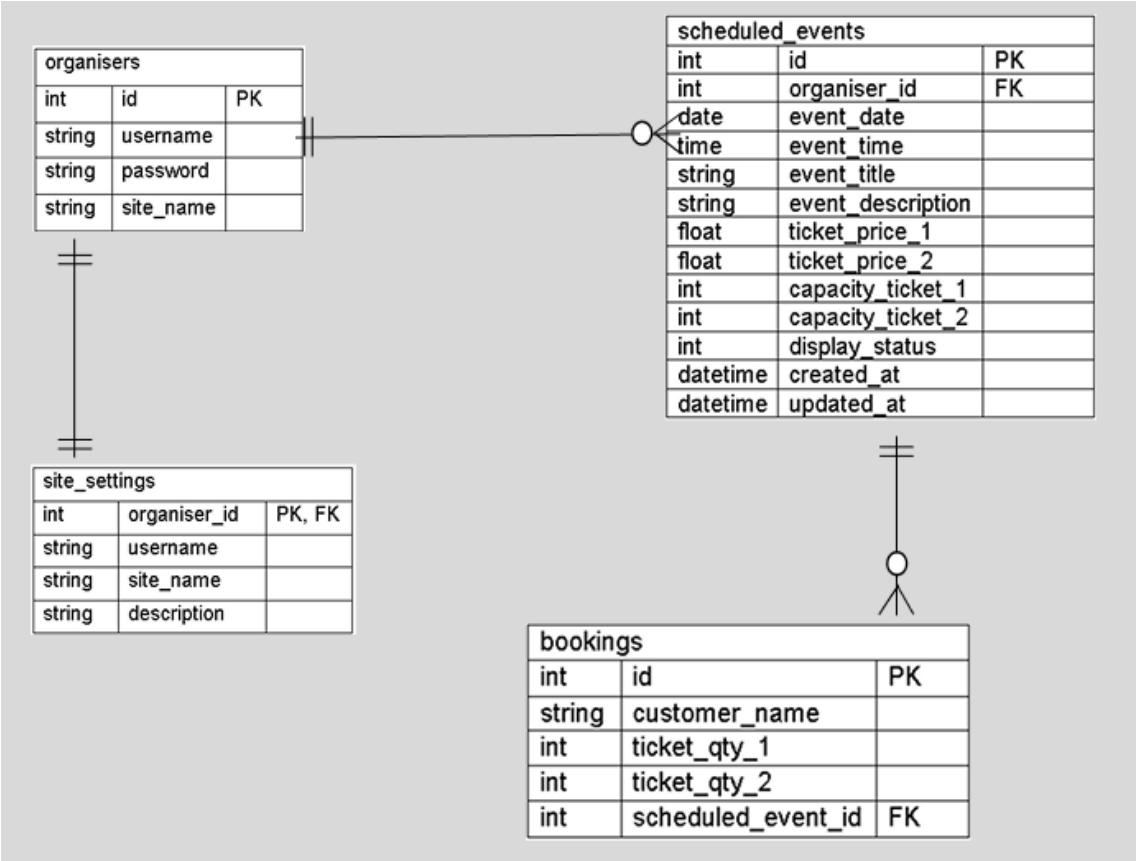

Figure 2 ER Diagram

Figure 2 shows a crow's feet ER diagram with cardinalities:

- organisers - site_settings (1:1, one to one), this means each organiser only have one site settings entry.
- organisers - scheduled_events (1:N, one to many), this means each organiser can have zero or more scheduled events.
- scheduled_events - bookings (1:N, one to many), this means each event can have zero or more bookings.

Tables site_settings, scheduled_events and bookings use ON DELETE CASCADE foreign keys to ensure removing an entry will remove all dependent records automatically, preventing orphan rows.

4. **Extension**: Authentication for Organiser Pages

The chosen extension is Authentication for Organiser Pages.

The extension is implemented using additional libraries, bcrypt and express-session. The session management is setup in index,js

Code Snippet (line 15-23):

```
// Set up session management
const session = require("express-session");
app.use(
  session({
    secret: "sql-project-secret",
    resave: false,
    saveUninitialized: true,
  })
);
```

This extension secures the organiser pages with password authentication and bcrypt to hash passwords to prevent unauthorized access. routes/auth.js handles login, logout and signup requests. On success, it stores organiser row in req.session.organiser. routes/organiser.js rejects any request lacking that session object.

Code Snippet (line 10-13):

```
router.use(function (req, res, next) {
  if (!req.session.organiser) return
res.redirect("/login_organiser");
  next();
});
```

View login_organiser.ejs renders a simple login form. The Organiser Home link now redirects to /login_organiser when no session exists.

In routes/auth.js,

- During signup, passwords are hashed with 10 salt rounds and stored in the organisers table.
- During login, the input is compared to the stored hash, on success, the organiser row is saved to req.session.organiser.
- logout destroy the session and redirect to organiser login page.

Code Snippet (line 58):

```
-        bcrypt.compare(password, organiser.password, function
(err2, result) {
```

Code Snippet (line 139):

```
        bcrypt.hash(password, 10, function (errHash, hash) {
```